Machine Coding: Auth Guard with Permission-Based Routes and Role-Based Redirection

Problem: Build an auth guard for a React app. Logged-out users can't open private pages. Logged-in users can only open pages their permissions allow. After login, each role lands on its own home: **admin** → **/admin**, **manager** → **/manager**, **user** → **/dashboard**. Buttons and nav links inside pages also respect permissions.

Tech stack: React (Vite, JavaScript) + react-router-dom v7 · **Time limit:** 60 minutes (default)

PART A: Thought process

Steps that work for any machine coding task:

1. **Understand the requirements:** functional + non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions:** every action and what should happen. This is your checklist.
3. **Design the architecture:** modules and who talks to whom.
4. **Design the state:** what you store, what you derive, who owns it.
5. **Design the contracts:** data shapes, API inputs and outputs.
6. **Trace one flow end to end** before writing code.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers:** the simplest working version first, then one layer at a time.
9. **Cover states and edge cases:** loading, empty, error, and the problem-specific ones.
10. **Wrap up:** trade-offs, what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases, architecture, state, contracts	8 min
Build: mock API + permissions + auth context	12 min
Build: guards + routes + role redirect	15 min
Build: login page, nav, `<Can>`, 403/404	10 min
Edge cases, polish, walkthrough	10 min

Think out loud. The interviewer grades your reasoning as much as your code. Say *why* before *what*: "I'll keep permissions as data, not `if (role === 'admin')` checks, because roles change and permissions don't."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Login with username + password; show an error on bad credentials.
- Public route: `/login`. Private routes: `/dashboard`, `/reports`, `/manager`, `/admin`, `/settings`.

- Logged-out user opening a private URL → redirect to `/login`, and after login go **back to that URL** (if allowed).
- Logged-in user opening a page without permission → **403** screen.
- Logged-in user opening `/login` → redirect away to their home.
- `/` sends each role to its home (admin → `/admin`, manager → `/manager`, user → `/dashboard`).
- Nav shows only links the user can open; buttons inside pages hide by permission (`<Can>`).
- Session survives a page refresh; logout clears it (in every open tab).
- Unknown URL → 404.

Non-functional requirements

Area	What it means here
Performance	Permission check is a tiny array lookup; no extra renders on every route change.
Scalability	Adding a role or page = editing data (`ROLE_PERMISSIONS`, `APP_ROUTES`), not adding `if` chains.
Error handling	Bad credentials, expired token, and failed session restore all end in a clean logged-out state.
Accessibility	Labeled inputs, `role="alert"` errors, `role="status"` loader, keyboard-friendly links/buttons, focus ring.
Reusability	`RequireAuth`, `GuestOnly`, `Can`, `useAuth` work for any page.
Extensibility	Swap the mock API for a real one without touching guards.
Security	Client guards are **UX only**; the server must check permissions on every API call.

Clarifying questions to ask

1. Roles fixed (admin/manager/user) or can there be custom roles? → Decides role checks vs permission checks.
2. Does the backend send permissions, or should the client map role → permissions?
3. Where is the token stored: httpOnly cookie or localStorage? Is there a refresh token?
4. Unauthorized page: show a 403 in place, or redirect to home?
5. Should a user be sent back to the page they tried to open after login?
6. Can one user have multiple roles?

Assumptions we will build with

- Three roles; one role per user. Guards check **permissions**, never role names (except for picking the home page).
- The (mock) server returns `user.permissions`; the client trusts that list for UI only.
- Token in `localStorage` (simple for the interview; cookie discussed in trade-offs).
- 403 is rendered in place (URL stays). After login we return to the original URL **only if** the user may open it, else to their role home.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Logged-out user opens `/reports`	Redirect to `/login`, remember `from=/reports`
Submit empty form	"Enter username and password", no API call
Submit wrong password	"Invalid username or password", button re-enabled
Submit while a request is in flight	Ignored (button disabled, guard in handler)
Manager logs in with `from=/reports`	Lands on `/reports`
User logs in with `from=/reports` (not allowed)	Lands on `/dashboard`, not on a 403
Admin logs in with no `from`	Lands on `/admin`

Action / input	Expected behavior / output
Logged-in user opens `/`	Redirect to role home
User opens `/admin`	403 screen, URL stays `/admin`
Logged-in user opens `/login`	Redirect to role home
Refresh on a private page	"Checking your session…" then the same page (no flash of login)
Stored token expired	Cleared, user sent to `/login`
Logout	Token removed, redirect to `/login`
Logout in another tab	This tab also logs out
Nav bar	Only links the user can open
Dashboard "Open reports" button	Shown only with `reports:view`
Unknown URL `/nope`	404 page

---

3. Design the architecture

Module	Responsibility
`permissions.js`	Permission constants, role → permissions, role → home, `hasPermission()`
`routeConfig.js`	Route list as data (path, label, permission), `redirectAfterLogin()`
`authApi.js`	Mock backend: `login()`, `getMe()`
`useAuth.js`	The context object + `useAuth()` hook
`AuthProvider.jsx`	Owns `user` + `status`, restores session, login/logout, cross-tab sync
`guards.jsx`	`RequireAuth`, `GuestOnly`, `RoleHomeRedirect`, `Can`, loader, 403
`pages.jsx`	Login, Dashboard, Reports, Manager, Admin, Settings, 404
`App.jsx`	Router tree + layout with permission-filtered nav

```
<BrowserRouter>
 └ <AuthProvider>               user, status, login, logout, can
    └ <Routes>
       ├ <GuestOnly>             logged in? → redirect away
       │   └ /login  <LoginPage>
       ├ <RequireAuth>           logged out? → /login (remember from)
       │   └ <Layout>            nav filtered by can()
       │       ├ /  <RoleHomeRedirect>
       │       ├ /dashboard  <RequireAuth permission="dashboard:view">
       │       ├ /reports    <RequireAuth permission="reports:view">
       │       ├ /manager    <RequireAuth permission="team:manage">
       │       ├ /admin      <RequireAuth permission="users:manage">
       │       └ /settings   <RequireAuth permission="settings:edit">
       └ *  <NotFoundPage>
```

Start simple: one context + one guard. Split `GuestOnly` and `Can` out only once you see the login page and the buttons need different behavior.

4. Design the state / data model

State or field	Purpose
`user`	`null` or `{ id, name, role, permissions }` from the server
`status`	`'checking' | 'authenticated' | 'anonymous'`; stops guards from redirecting before the session is restored
`localStorage.auth_token`	Survives refresh; read once on boot
`location.state.from`	The URL the user tried to open before login (router owns it)
Login form: `form`, `error`, `submitting`	Local to `LoginPage`

```
const [user, setUser] = useState(null);
const [status, setStatus] = useState(() => (readToken() ? 'checking' : 'anonymous'));
```

Senior note

- **Derived, not stored:** `isAuthenticated` (= `!!user`), `can(permission)`, the role home, the visible nav links, and the post-login redirect are all computed from `user`. Storing them would let them drift out of sync.
- **Ownership:** `AuthProvider` owns the session. The router owns "where the user was going" (`location.state.from`). The login form owns its own inputs. No page keeps its own copy of the user.
- `status` exists because `user === null` means two different things: "not logged in" and "not known yet". Mixing them causes the classic refresh-redirect bug (Challenge 1).

---

5. Design the data structures and contracts

Permissions as data

```
export const PERMISSIONS = {
  DASHBOARD_VIEW: 'dashboard:view',
  REPORTS_VIEW: 'reports:view',
  TEAM_MANAGE: 'team:manage',
  USERS_MANAGE: 'users:manage',
  SETTINGS_EDIT: 'settings:edit',
};

export const ROLE_PERMISSIONS = {
  admin: Object.values(PERMISSIONS),
  manager: ['dashboard:view', 'reports:view', 'team:manage'],
  user: ['dashboard:view'],
};

export const ROLE_HOME = { admin: '/admin', manager: '/manager', user: '/dashboard' };
```

Why `resource:action` strings: readable, greppable, and the same strings the backend uses. Why stable user ids (`u1`): the token and `getMe()` look the user up by id, not by name.

Route config as data (one list feeds the router, the nav and the redirect check):

```
{ path: '/reports', label: 'Reports', permission: 'reports:view' }
```

API contracts (mocked)

Call	Input	Success	Failure
`login(username, password)`	`'manager', 'manager123'`	`{ token, user }`	throws `Error('Invalid username or password')`
`getMe(token)`	`'u2.1767000000000'`	`user`	throws `{ status: 401, message: 'Session expired' }`

```
{
  "token": "u2.1767000000000",
  "user": {
    "id": "u2",
    "name": "Ravi (Manager)",
    "role": "manager",
    "permissions": ["dashboard:view", "reports:view", "team:manage"]
  }
}
```

The server never returns the password, and it attaches `permissions` so the client never has to guess.

---

6. Trace the data flow

Logged-out manager opens `/reports`, logs in, and lands back on `/reports`:

```
URL /reports
  ↓
<RequireAuth>  status='anonymous', user=null
  ↓
<Navigate to="/login" state={{ from: location }} replace>
  ↓
<GuestOnly> user=null → render <LoginPage>
  ↓
submit → setSubmitting(true) → authApi.login('manager', '…')   (500 ms)
  ↓
AuthProvider.login: localStorage.setItem(token); setUser(me); setStatus('authenticated')
  ↓
<GuestOnly> re-renders: user exists
  ↓
redirectAfterLogin(me, state.from)
  → canAccessPath(me, '/reports') → hasPermission(me, 'reports:view') → true
  ↓
<Navigate to="/reports" replace>
  ↓
<RequireAuth> ok → <Layout> (nav: Dashboard, Reports, Team) → <RequireAuth permission="reports:view"> ok →
<ReportsPage>
```

---

7. Important technical challenges

Challenge 1: Refresh sends a logged-in user to /login

Problem: User refreshes on `/admin`. On the first render `user` is `null` (session not restored yet), so the guard redirects to `/login`. Then `getMe()` resolves, and the user lands on their home instead of `/admin`. There's also a visible flash of the login page.

Solution: A third state, `'checking'`. Start in `'checking'` if a token exists; guards render a loader instead of deciding.

```
const [status, setStatus] = useState(() => (readToken() ? 'checking' : 'anonymous'));

export function RequireAuth({ permission, children }) {
  const { status, user, can } = useAuth();
  const location = useLocation();
  if (status === 'checking') return <FullPageLoader />;
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  if (permission && !can(permission)) return <Forbidden />;
  return children ?? <Outlet />;
}
```

Challenge 2: Redirect after login, without landing on a 403

Problem: A plain user clicks a shared link to `/reports`, logs in, and gets "403". Bad experience. Also, if both `LoginPage` (`navigate(from)`) and `GuestOnly` (`<Navigate to={home}>`) redirect, they race and the user ends up on the wrong page.

Solution: Exactly one place decides: `GuestOnly`. It sends the user to `from` only if they can open it, else to their role home. `LoginPage` just calls `login()`.

```
export function redirectAfterLogin(user, from) {
  return from && canAccessPath(user, from.pathname)
    ? from.pathname + (from.search ?? '')
    : homeFor(user);
}

// GuestOnly
if (user) return <Navigate to={redirectAfterLogin(user, location.state?.from)} replace />;
```

`from` lives in router state, not in a `?redirect=` query param, so an attacker can't craft a link that redirects to another site (open redirect).

Challenge 3: Permissions vs roles, and one source of truth

Problem: Code like `if (user.role === 'admin' || user.role === 'manager')` gets copied into routes, nav and buttons. A new role "auditor" means editing every one of them, and one gets missed.

Solution: Check **permissions** everywhere; roles only map to permissions and a home page. The same `APP_ROUTES` list drives the router, the nav and the redirect check.

```
export function hasPermission(user, required) {
  if (!user) return false;
  if (!required) return true;
  const list = Array.isArray(required) ? required : [required];
  return list.every((p) => user.permissions.includes(p));
}

// nav
```

```
APP_ROUTES.filter((r) => can(r.permission)).map((r) => <NavLink to={r.path}>{r.label}</NavLink>);

// button inside a page
<Can permission="reports:view"><Link to="/reports">Open reports</Link></Can>
```

Challenge 4: Session lifecycle across tabs and expiry

Problem: User logs out in tab A, tab B still shows the admin page and keeps calling APIs. Or the token expires while the app is open.

Solution: Listen to the `storage` event (fires in *other* tabs) and drop the session. On boot, an expired token fails `getMe()` and is cleared. (With more time: a fetch wrapper that calls `logout()` on any 401.)

```
useEffect(() => {
  const onStorage = (e) => {
    if (e.key === TOKEN_KEY && !e.newValue) {
      setUser(null);
      setStatus('anonymous');
    }
  };
  window.addEventListener('storage', onStorage);
  return () => window.removeEventListener('storage', onStorage);
}, []);
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite app + `react-router-dom`, 2 dummy pages, `<BrowserRouter>`	Routing works
2	`permissions.js`: constants, role map, `hasPermission`, `homeFor`	Pure logic, testable
3	`authApi.js` mock `login()` / `getMe()` with delay	Fake backend
4	`AuthProvider` + `useAuth`: `user`, `login`, `logout`	Can log in/out
5	`RequireAuth` (auth only) + `/login` page	Private pages blocked
6	Remember `from`, `GuestOnly` + `redirectAfterLogin`	Back to the requested page
7	`status: 'checking'` + token restore on refresh	Refresh keeps you in
8	`RequireAuth permission=…` + 403 + `APP_ROUTES`	Permission routes
9	`RoleHomeRedirect` on `/`	admin/manager/user homes
10	Layout nav filtered by `can()`, `<Can>` for buttons	UI matches permissions
11	Login form states (validation, error, submitting), 404	Polished UX
12	Cross-tab logout, a11y pass, edge cases	Done

---

9. Cover all states and edge cases

State / edge case	Handling
Initial, no token	`status='anonymous'`, private URL → `/login`
Initial, token present	`status='checking'`, loader until `getMe()`
Login submitting	Button disabled, "Signing in…", double submit ignored
Login success	`GuestOnly` redirects (from or role home)
Login empty fields	Inline error, no request
Login wrong credentials	`role="alert"` error, form usable again
Expired / tampered token	`getMe()` fails → token cleared → `/login`
`localStorage` blocked (private mode)	`readToken()` try/catch → treated as logged out
No permission for a route	403 in place, link to role home
`from` not allowed for the new user	Role home instead of 403
`from` is `/login` or unknown	Role home
Logged-in user visits `/login`	Redirect to role home
Unknown role from server	`homeFor()` falls back to `/login`; `can()` false for everything
Logout	Token removed → outer guard → `/login` (remembering the page)
Logout in another tab	`storage` event → logged out here too
Unknown URL	404
StrictMode double effect	`cancelled` flag ignores the stale `getMe()` result

---

10. Final structure

```
_58_auth_guard_permission_based_routing/
├─ index.html
├─ package.json            react, react-dom, react-router-dom
└─ src/
   ├─ main.jsx
   ├─ App.jsx              router tree + Layout (nav)
   ├─ App.css
   ├─ pages.jsx            Login, Dashboard, Reports, Manager, Admin, Settings, 404
   └─ auth/
      ├─ permissions.js    PERMISSIONS, ROLE_PERMISSIONS, ROLE_HOME, hasPermission, homeFor
      ├─ routeConfig.js    APP_ROUTES, canAccessPath, redirectAfterLogin
      ├─ authApi.js        mock login / getMe
      ├─ useAuth.js        AuthContext + useAuth
      ├─ AuthProvider.jsx  session state, restore, login/logout, cross-tab
      └─ guards.jsx        RequireAuth, GuestOnly, RoleHomeRedirect, Can, 403, loader
```

`useAuth.js` is separate from `AuthProvider.jsx` so each file exports only components or only non-components (keeps Vite fast refresh working).

---

11. Full working code

Install: `npm i react-router-dom`. Demo logins: `admin / admin123`, `manager / manager123`, `user / user123`.

src/auth/permissions.js

```
// Single source of truth for what each role may do.
export const PERMISSIONS = {
  DASHBOARD_VIEW: 'dashboard:view',
  REPORTS_VIEW: 'reports:view',
  TEAM_MANAGE: 'team:manage',
  USERS_MANAGE: 'users:manage',
  SETTINGS_EDIT: 'settings:edit',
};

const P = PERMISSIONS;

export const ROLE_PERMISSIONS = {
  admin: Object.values(P),
  manager: [P.DASHBOARD_VIEW, P.REPORTS_VIEW, P.TEAM_MANAGE],
  user: [P.DASHBOARD_VIEW],
};

// Where each role lands after login or on "/".
export const ROLE_HOME = {
  admin: '/admin',
  manager: '/manager',
  user: '/dashboard',
};

// required: one permission or an array (all must be granted).
export function hasPermission(user, required) {
  if (!user) return false;
  if (!required) return true;
  const list = Array.isArray(required) ? required : [required];
  return list.every((p) => user.permissions.includes(p));
}

export const homeFor = (user) => ROLE_HOME[user?.role] ?? '/login';
```

src/auth/routeConfig.js

```
import { PERMISSIONS as P, hasPermission, homeFor } from './permissions.js';

// Data only (no JSX): guards, nav and redirect logic all read this list.
export const APP_ROUTES = [
  { path: '/dashboard', label: 'Dashboard', permission: P.DASHBOARD_VIEW },
  { path: '/reports', label: 'Reports', permission: P.REPORTS_VIEW },
  { path: '/manager', label: 'Team', permission: P.TEAM_MANAGE },
  { path: '/admin', label: 'Admin', permission: P.USERS_MANAGE },
  { path: '/settings', label: 'Settings', permission: P.SETTINGS_EDIT },
];

// shortcut: exact path match only, switch to matchPath() once routes have params.
export function canAccessPath(user, pathname) {
```

```
  const route = APP_ROUTES.find((r) => r.path === pathname);
  return Boolean(route) && hasPermission(user, route.permission);
}

// Send the user back to the page they asked for, but only if they may open it.
export function redirectAfterLogin(user, from) {
  return from && canAccessPath(user, from.pathname)
    ? from.pathname + (from.search ?? '')
    : homeFor(user);
}
```

src/auth/authApi.js

```
import { ROLE_PERMISSIONS } from './permissions.js';

// Mock backend. In a real app the server owns users, roles and permissions.
const USERS = [
  { id: 'u1', username: 'admin', password: 'admin123', name: 'Asha (Admin)', role: 'admin' },
  { id: 'u2', username: 'manager', password: 'manager123', name: 'Ravi (Manager)', role: 'manager' },
  { id: 'u3', username: 'user', password: 'user123', name: 'Neha (User)', role: 'user' },
];

const TOKEN_TTL_MS = 30 * 60 * 1000;
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// Never send the password back; the server attaches the permission list.
const toPublicUser = ({ id, name, role }) => ({ id, name, role, permissions: ROLE_PERMISSIONS[role] });

export async function login(username, password) {
  await delay(500);
  const user = USERS.find((u) => u.username === username.trim() && u.password === password);
  if (!user) throw new Error('Invalid username or password');
  return { token: `${user.id}.${Date.now() + TOKEN_TTL_MS}`, user: toPublicUser(user) };
}

export async function getMe(token) {
  await delay(400);
  const [id, expiresAt] = String(token).split('.');
  const user = USERS.find((u) => u.id === id);
  if (!user || Number(expiresAt) < Date.now()) {
    const error = new Error('Session expired');
    error.status = 401;
    throw error;
  }
  return toPublicUser(user);
}
```

src/auth/useAuth.js

```
import { createContext, useContext } from 'react';

export const AuthContext = createContext(null);

export function useAuth() {
```

```
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
```

src/auth/AuthProvider.jsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { getMe, login as apiLogin } from './authApi.js';
import { hasPermission } from './permissions.js';
import { AuthContext } from './useAuth.js';

const TOKEN_KEY = 'auth_token';

const readToken = () => {
  try {
    return localStorage.getItem(TOKEN_KEY);
  } catch {
    return null;
  }
};

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  // 'checking' while a stored token is verified, so guards don't redirect too early.
  const [status, setStatus] = useState(() => (readToken() ? 'checking' : 'anonymous'));

  const clearSession = useCallback(() => {
    localStorage.removeItem(TOKEN_KEY);
    setUser(null);
    setStatus('anonymous');
  }, []);

  // Restore the session on page load / refresh.
  useEffect(() => {
    const token = readToken();
    if (!token) return;
    let cancelled = false;
    getMe(token)
      .then((me) => {
        if (cancelled) return;
        setUser(me);
        setStatus('authenticated');
      })
      .catch(() => {
        if (!cancelled) clearSession();
      });
    return () => {
      cancelled = true;
    };
  }, [clearSession]);

  // Logout in one tab logs out every tab.
  useEffect(() => {
    const onStorage = (e) => {
```

```
      if (e.key === TOKEN_KEY && !e.newValue) {
        setUser(null);
        setStatus('anonymous');
      }
    };
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
  }, []);

  const login = useCallback(async (username, password) => {
    const { token, user: me } = await apiLogin(username, password);
    localStorage.setItem(TOKEN_KEY, token);
    setUser(me);
    setStatus('authenticated');
    return me;
  }, []);

  const can = useCallback((permission) => hasPermission(user, permission), [user]);

  const value = useMemo(
    () => ({ user, status, login, logout: clearSession, can }),
    [user, status, login, clearSession, can],
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}
```

src/auth/guards.jsx

```
import { Link, Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from './useAuth.js';
import { homeFor } from './permissions.js';
import { redirectAfterLogin } from './routeConfig.js';

export function FullPageLoader() {
  return (
    <div className="center" role="status" aria-live="polite">
      Checking your session…
    </div>
  );
}

export function Forbidden() {
  const { user } = useAuth();
  return (
    <section className="card">
      <h1>403 · Access denied</h1>
      <p>
        Your role <b>{user.role}</b> cannot open this page.
      </p>
      <Link className="btn" to={homeFor(user)}>
        Go to my home
      </Link>
    </section>
  );
```

```
}

// Must be logged in; optionally must also have `permission`.
export function RequireAuth({ permission, children }) {
  const { status, user, can } = useAuth();
  const location = useLocation();

  if (status === 'checking') return <FullPageLoader />;
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  // Render 403 in place (URL stays) instead of redirecting, so the user sees what was blocked.
  if (permission && !can(permission)) return <Forbidden />;
  return children ?? <Outlet />;
}

// Only for logged-out users (login page). Logged-in users go where they were heading.
export function GuestOnly() {
  const { status, user } = useAuth();
  const location = useLocation();

  if (status === 'checking') return <FullPageLoader />;
  if (user) return <Navigate to={redirectAfterLogin(user, location.state?.from)} replace />;
  return <Outlet />;
}

// "/" → role home: admin → /admin, manager → /manager, user → /dashboard.
export function RoleHomeRedirect() {
  const { user } = useAuth();
  return <Navigate to={homeFor(user)} replace />;
}

// UI-level guard for buttons / sections inside a page.
export function Can({ permission, fallback = null, children }) {
  const { can } = useAuth();
  return can(permission) ? children : fallback;
}
```

src/pages.jsx

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from './auth/useAuth.js';
import { Can } from './auth/guards.jsx';
import { PERMISSIONS as P } from './auth/permissions.js';

const DEMO_USERS = [
  { username: 'admin', password: 'admin123' },
  { username: 'manager', password: 'manager123' },
  { username: 'user', password: 'user123' },
];

export function LoginPage() {
  const { login } = useAuth();
  const [form, setForm] = useState({ username: '', password: '' });
  const [error, setError] = useState('');
  const [submitting, setSubmitting] = useState(false);
```

```
const onChange = (e) => setForm((f) => ({ ...f, [e.target.name]: e.target.value }));

// No navigate() here: once `user` is set, <GuestOnly> redirects (one place owns redirects).
const onSubmit = async (e) => {
  e.preventDefault();
  if (submitting) return;
  if (!form.username.trim() || !form.password) {
    setError('Enter username and password');
    return;
  }
  setError('');
  setSubmitting(true);
  try {
    await login(form.username, form.password);
  } catch (err) {
    setError(err.message);
    setSubmitting(false);
  }
};

return (
  <main className="center">
    <form className="card login" onSubmit={onSubmit} noValidate>
      <h1>Sign in</h1>
      <label>
        Username
        <input name="username" value={form.username} onChange={onChange} autoComplete="username" />
      </label>
      <label>
        Password
        <input
          name="password"
          type="password"
          value={form.password}
          onChange={onChange}
          autoComplete="current-password"
        />
      </label>
      {error && (
        <p className="error" role="alert">
          {error}
        </p>
      )}
      <button className="btn" type="submit" disabled={submitting}>
        {submitting ? 'Signing in…' : 'Sign in'}
      </button>
      <div className="demo">
        Demo:
        {DEMO_USERS.map((u) => (
          <button type="button" key={u.username} onClick={() => setForm(u)}>
            {u.username}
          </button>
        ))}
      </div>
    </form>
```

```
    </main>
  );
}

export function DashboardPage() {
  const { user } = useAuth();
  return (
    <section className="card">
      <h1>Dashboard</h1>
      <p>Welcome, {user.name}. Everyone with dashboard:view sees this page.</p>
      <Can permission={P.REPORTS_VIEW} fallback={<p className="muted">Reports are hidden for your role.</p>}>
        <Link className="btn" to="/reports">
          Open reports
        </Link>
      </Can>
    </section>
  );
}

export function ReportsPage() {
  return (
    <section className="card">
      <h1>Reports</h1>
      <p>Visible to manager and admin.</p>
    </section>
  );
}

export function ManagerPage() {
  return (
    <section className="card">
      <h1>Team (Manager home)</h1>
      <p>Manage your team. Needs team:manage.</p>
      <Can permission={P.USERS_MANAGE}>
        <button className="btn" type="button">
          Promote member (admin only)
        </button>
      </Can>
    </section>
  );
}

export function AdminPage() {
  return (
    <section className="card">
      <h1>Admin (Admin home)</h1>
      <p>Manage all users. Needs users:manage.</p>
    </section>
  );
}

export function SettingsPage() {
  return (
    <section className="card">
      <h1>Settings</h1>
      <p>Needs settings:edit (admin only).</p>
```

```
    </section>
  );
}

export function NotFoundPage() {
  return (
    <main className="center">
      <section className="card">
        <h1>404 · Page not found</h1>
        <Link className="btn" to="/">
          Go home
        </Link>
      </section>
    </main>
  );
}
```

src/App.jsx

```
import { BrowserRouter, NavLink, Outlet, Route, Routes } from 'react-router-dom';
import { AuthProvider } from './auth/AuthProvider.jsx';
import { useAuth } from './auth/useAuth.js';
import { GuestOnly, RequireAuth, RoleHomeRedirect } from './auth/guards.jsx';
import { APP_ROUTES } from './auth/routeConfig.js';
import {
  AdminPage,
  DashboardPage,
  LoginPage,
  ManagerPage,
  NotFoundPage,
  ReportsPage,
  SettingsPage,
} from './pages.jsx';
import './App.css';

const PAGES = {
  '/dashboard': <DashboardPage />,
  '/reports': <ReportsPage />,
  '/manager': <ManagerPage />,
  '/admin': <AdminPage />,
  '/settings': <SettingsPage />,
};

function Layout() {
  const { user, can, logout } = useAuth();
  return (
    <div className="layout">
      <header>
        <nav aria-label="Main">
          {/* Nav is filtered by the same permissions the routes use. */}
          {APP_ROUTES.filter((r) => can(r.permission)).map((r) => (
            <NavLink key={r.path} to={r.path}>
              {r.label}
            </NavLink>
          ))}
```

```
        </nav>
        <div className="who">
          <span>
            {user.name} <span className="badge">{user.role}</span>
          </span>
          <button className="btn" type="button" onClick={logout}>
            Logout
          </button>
        </div>
      </header>
      <main>
        <Outlet />
      </main>
    </div>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          <Route element={<GuestOnly />}>
            <Route path="/login" element={<LoginPage />} />
          </Route>

          <Route element={<RequireAuth />}>
            <Route element={<Layout />}>
              <Route index element={<RoleHomeRedirect />} />
              {APP_ROUTES.map((r) => (
                <Route
                  key={r.path}
                  path={r.path}
                  element={<RequireAuth permission={r.permission}>{PAGES[r.path]}</RequireAuth>}
                />
              ))}
            </Route>
          </Route>

          <Route path="*" element={<NotFoundPage />} />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
```

```
  </StrictMode>,
);
```

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f4f5f7; color: #1f2328; }
.center { min-height: 100vh; display: grid; place-items: center; padding: 16px; }
.card { background: #fff; border: 1px solid #d8dee4; border-radius: 10px; padding: 24px; max-width: 520px; }
.login { width: 100%; max-width: 360px; display: grid; gap: 12px; }
.login label { display: grid; gap: 4px; font-size: 14px; }
.login input { padding: 8px 10px; border: 1px solid #d0d7de; border-radius: 6px; font: inherit; }
.btn { display: inline-block; padding: 8px 14px; border: 0; border-radius: 6px; background: #2563eb; color:
#fff; font: inherit; cursor: pointer; text-decoration: none; }
.btn:disabled { opacity: 0.6; cursor: not-allowed; }
.error { color: #b42318; margin: 0; }
.muted { color: #6e7781; }
.demo { display: flex; gap: 6px; align-items: center; font-size: 13px; }
.layout header { display: flex; flex-wrap: wrap; gap: 12px; justify-content: space-between; align-items: center;
padding: 12px 16px; background: #fff; border-bottom: 1px solid #d8dee4; }
.layout nav { display: flex; gap: 12px; flex-wrap: wrap; }
.layout nav a { color: #57606a; text-decoration: none; }
.layout nav a.active { color: #2563eb; font-weight: 600; }
.layout main { padding: 16px; }
.who { display: flex; gap: 10px; align-items: center; }
.badge { background: #eef2ff; color: #3730a3; border-radius: 999px; padding: 2px 8px; font-size: 12px; }
:focus-visible { outline: 2px solid #2563eb; outline-offset: 2px; }
```

---

12. Wrap-up

Edge cases handled: refresh without login flash, deep link → login → back, no 403 after login, logged-in user on `/login`, expired/tampered token, blocked storage, cross-tab logout, double submit, unknown role, 404, StrictMode double effects.

Trade-offs to mention

- **Client guard ≠ security.** Anyone can edit JS. The server must check the permission on every API call; the client guard only keeps the UI honest.
- **localStorage vs httpOnly cookie:** localStorage is readable by any XSS script; an httpOnly + SameSite cookie is safer but needs CSRF protection and a `/me` endpoint. I used localStorage to keep the interview version short.
- **403 in place vs redirect:** in place keeps the URL (clear what was blocked, works after a role upgrade + refresh). Redirecting hides the URL and can confuse.
- **Permission list from server vs role map on client:** server list = one source of truth; the client map is only used by the mock.
- **Wrapper components vs route `loader`s:** wrappers are simple and readable; data-router loaders can redirect before render (no loader flash) but need `createBrowserRouter`.

With more time

- `apiFetch` wrapper: attach token, on 401 call `logout()`; silent refresh with a refresh token.
- `matchPath` for routes with params (`/projects/:id`), plus resource-level checks (can edit *this* project).
- Lazy-load pages per role (`React.lazy`) so a user never downloads admin code.
- Tests: unit tests for `hasPermission` / `redirectAfterLogin`; React Testing Library for each guard with a `MemoryRouter`.
- Session-expiry warning modal, idle timeout, audit of 403 hits.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why check permissions instead of roles?

Roles are groups that change ("manager can now see settings"); permissions are the actual abilities. Checking permissions means a role change is one edit in `ROLE_PERMISSIONS`, and a new role needs no code changes in routes, nav or buttons.

Q2. Is the client-side guard enough to protect admin data?

No. It's UX only; the JS bundle and `localStorage` can be edited. The API must verify the token and permission for every request. If the client guard is bypassed, the user just sees an empty admin page with failing requests.

Q3. What if a user can have multiple roles?

The server merges permissions from all roles into one `permissions` list, so `hasPermission` doesn't change. For the home page, pick by priority: `['admin', 'manager', 'user'].find((r) => user.roles.includes(r))`.

State design

Q4. Why a `status` field? Isn't `user === null` enough?

`null` means both "logged out" and "not loaded yet". Without `'checking'`, a refresh redirects to `/login` before `getMe()` finishes. Three states make the guard's decision unambiguous.

Q5. Why Context and not Redux / Zustand?

The auth state is small, changes rarely (login/logout), and is read by many components: the textbook use case for Context. If the app already uses Redux, I'd put it in a slice; the guards would only change their hook.

Q6. Won't every context consumer re-render on each route change?

No. The context value is memoized with `useMemo` and only changes when `user` or `status` changes, which happens at login/logout, not on navigation.

Challenge: redirects

Q7. Why does `LoginPage` not call `navigate()` after login?

Because `GuestOnly` will also react to `user` being set. Two redirects race and the wrong one can win. One owner (`GuestOnly` + `redirectAfterLogin`) keeps the rule in one place.

Q8. Why `replace` on every `<Navigate>`?

So the Back button doesn't return to a page that immediately redirects again (redirect loop in history). After login, Back shouldn't reopen the login form.

Q9. Why keep `from` in router state, not `?redirect=/admin`?

A query param can be crafted: `/login?redirect=https://evil.com`. Router state can't be set from a link. If you must use a query param (e.g. it has to survive a full reload), only accept internal paths that start with `/` and are in `APP_ROUTES`.

Q10. Why render 403 instead of redirecting to home?

The user sees why they can't open the page and the URL stays, so after an admin grants access a refresh just works. Redirecting silently looks like a bug.

Challenge: session

Q11. What if the token expires while the user is on a page?

The next API call returns 401. A shared `apiFetch` wrapper calls `logout()`; the outer `RequireAuth` then redirects to `/login` with `from` set, so after re-login they return to the same page.

Q12. How would you sync login across tabs, not just logout?

In the `storage` handler, when `newValue` is a new token, call `getMe(newValue)` and set the user. Or use `BroadcastChannel('auth')` to post `{ type: 'login', user }` directly.

Q13. localStorage or httpOnly cookie?

Cookie (`httpOnly`, `Secure`, `SameSite=Lax`) is safer: XSS can't read it. Then the client never sees the token; on boot it calls `/me` to get the user and permissions. You add CSRF protection for state-changing requests.

Implementation steps

Q14. How do you support routes with params like `/projects/:id`?

Use `matchPath(r.path, pathname)` in `canAccessPath` instead of `===`. For "can edit *this* project", the route permission isn't enough: check ownership in the page (or the server returns `canEdit` per resource).

Q15. How do you hide a button but still explain why?

`<Can permission="x" fallback={<button disabled title="Ask an admin">Delete</button>}>`. The `fallback` prop already supports it.

Q16. How would you test the guards?

Render `<MemoryRouter initialEntries={['/admin']}>` with a fake `AuthContext.Provider` value per role. Assert: no user → login text; user role → 403; admin → admin page; `status='checking'` → loader. `hasPermission` and `redirectAfterLogin` are pure functions: plain unit tests.

Performance

Q17. What if there are 500 permissions and 200 routes?

Store `permissions` as a `Set` once when the user loads, so `has()` is O(1). Build `APP_ROUTES` into a `Map` by path. Both are computed once per login, not per render.

Q18. Admin code is large. Why should a plain user download it?

They shouldn't. `const AdminPage = lazy(() => import('./AdminPage'))` inside `<Suspense>`. The guard runs before the lazy element renders, so a user without `users:manage` never fetches that chunk.

Trade-offs / "why not X"

Q19. Why not use React Router `loader` / `redirect()` for guards?

Loaders can redirect before render (no loader flash, data fetched in parallel), which is great with `createBrowserRouter`. Wrapper components are simpler to explain in 60 minutes and work with `<BrowserRouter>`. I'd move to loaders in a real data-router app.

Q20. What if the API is slow and the session check takes 5 seconds?

Show the loader (already there), and render an app shell (header skeleton) instead of a blank screen. Optionally cache the last user in `sessionStorage` to paint optimistically, but still verify with `/me` and never trust the cache for API calls.

Q21. What if the permissions of a logged-in user change on the server?

The client keeps stale permissions until the next `getMe()`. Re-fetch `/me` on window focus or on a 403 from the API, or push the change over a WebSocket. The server is still safe because it checks on every request.